
2026 年（第十三届）英特尔杯大学生电子设计竞赛嵌入式 AI 专题赛

2026 Intel Cup Undergraduate Electronic Design Contest

- Embedded System Design Invitational Contest

作品设计报告

Final Report



Intel Cup Embedded System Design Contest

DESKTOP ROBOTIC ARM PICK-AND-PLACE SYSTEM BASED ON SWITCHABLE CLOUD/LOCAL MODELS

学生姓名: Ziheng Zhu

Xiangyu Bian

Pengcheng Li

指导教师: Xuemin Liu

参赛学校: 南京理工大学

Abstract

Huiyu Lingbi is a desktop embodied-intelligence robotic-arm system designed for natural-language-controlled picking, placement, and stacking. The architecture combines an Intel Core Ultra control platform, a JetArm six-servo manipulator, a Gemini Pro Plus camera, switchable cloud and local model backends, and an RGB visual closed loop. A model backend interprets the user request but may only return a structured task. Schema validation, object-ID checks, action allowlists, joint limits, motion generation, and fault handling remain local, preventing uncertain model output from directly controlling the servo bus. The design input is 600 x 480 RGB video at 60 fps. During grasping and placement, the controller repeatedly detects the target and gripper, executes bounded joint increments, and verifies results using J6 position feedback and multi-frame vision. The current repository implements bus-servo communication, joint configuration, forward kinematics, a numerical Jacobian, Windows and Ubuntu operation terminals, ROS 2 visualization, dry-run control, and safety stops. Fifty-eight hardware-free software tests pass. Camera calibration, perception integration, model adapters, and full closed-loop hardware validation remain in progress; therefore the specified accuracy and timing values are reported as acceptance targets rather than measured results.

Keywords: Intel Core Ultra; embodied intelligence; robotic arm; local model; RGB vision; visual closed-loop control.

1. Design Scope and Requirements

The system targets tabletop colored blocks, marked placement regions, and simple stacking tasks. A user specifies the object and spatial relation without entering servo IDs, joint angles, or Cartesian coordinates. The controller decomposes the request into observation, target selection, alignment, grasping, lifting, destination alignment, release, verification, and return-home states. Every task starts from the latest RGB frame and actual servo feedback; stale target coordinates are not reused.

The performance values below define the final acceptance envelope. They intentionally do not imply that hardware tests have already met the targets. Detection accuracy is defined primarily as precision, while gripper and TCP errors are mean Euclidean errors against traceable reference measurements. Grasp and placement convergence must be timed separately.

Metric	Design target	Acceptance interpretation
RGB input	600 x 480, 60 fps	60-s acquisition; report actual frame rate and drops
Object detection	Precision > 95%	Also report recall under lighting, occlusion, and background changes
Gripper localization	Mean error < 2 cm	Calibrated board or external measurement as ground truth
TCP localization	Mean error < 2 cm	Repeated XYZ measurements across the reachable workspace
Visual-loop convergence	< 3 s	Report grasp and placement loops independently
End-to-end task time	< 60 s	Instruction confirmation to verified result and return home

Evidence statement. *At the time of this report, only software and simulated-control results are available; hardware accuracy, frame-rate, and task-success measurements are pending.*

2. System Principle and Architecture

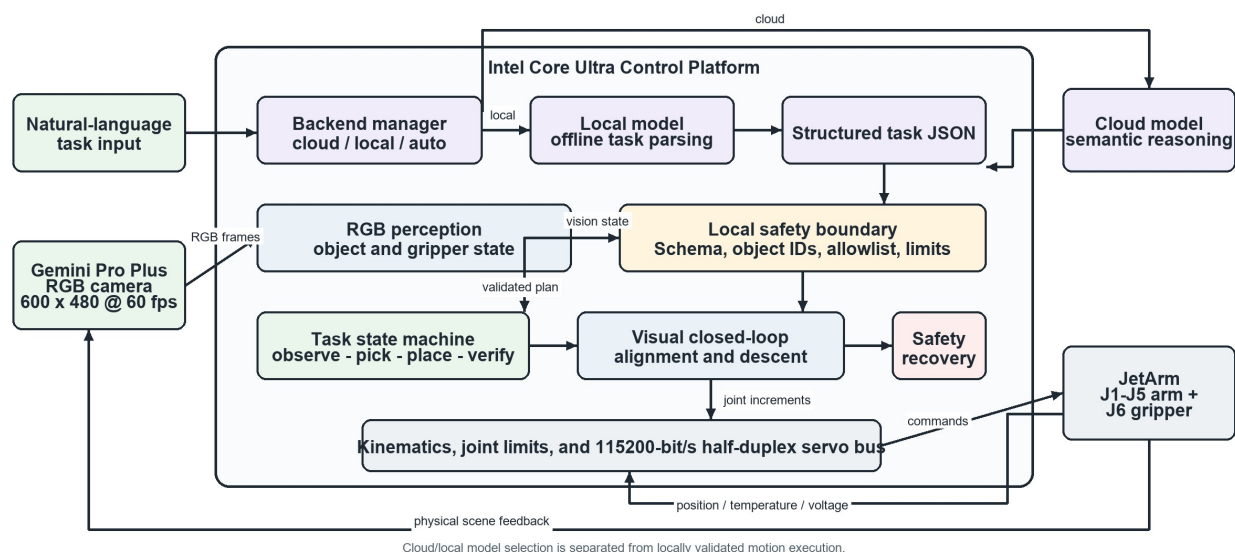


Figure 1. System architecture and locally validated closed-loop information flow.

2.1 Model Boundary and Task Representation

The backend manager exposes one task-parsing interface with cloud, local, and automatic modes. The cloud backend is intended for complex visual-semantic reasoning; the local backend provides offline operation, a predictable privacy boundary, and reduced network dependence. Automatic mode selects a primary backend before motion starts and uses a fallback only if the primary call fails, times out, or returns invalid structure. The selected backend is locked for the active task so execution logic cannot silently change mid-motion.

Model output contains a task ID, task type, target object ID, optional destination object ID, ordered steps, success conditions, and risk checks. It cannot contain raw servo positions, UART frames, or executable code. A local validator checks the JSON schema, verifies that referenced objects exist in the current vision state, accepts only allowlisted actions, and rejects plans that violate joint, table-height, or workspace constraints.

2.2 Perception, Planning, and Feedback

The RGB perception layer timestamps each frame and is designed to output object boxes, centers, confidence, left and right gripper tips, the gripper center and axis, opening width, and visibility. Semantic models may propose an object, but local low-latency detection tracks the selected target during motion. The task state machine converts the validated plan into finite states, while the visual controller converts image error into bounded J1-J4 increments. After each increment the system waits for a fresh frame and recomputes the error.

Servo feedback closes a second loop. J1-J4 use position commands; J5 controls wrist roll in speed mode; J6 controls the gripper in speed mode. Position, temperature, and voltage are read through the half-duplex bus. Continuous speed commands are stopped on mouse release, window close, explicit stop, timeout, or fault. Vision and servo feedback must agree before a task state is declared successful.

3. Hardware and Software Implementation

3.1 Hardware Chain

The intended hardware chain comprises an Intel DK2500 or equivalent Intel Core Ultra host, a Gemini Pro Plus camera mounted near J5/J6 in an eye-in-hand configuration, a USB-to-UART or servo-controller interface, a JetArm manipulator, and a dedicated servo power supply. The camera supplies RGB frames over USB. The host

communicates with all servos at 115200 bit/s over a shared half-duplex UART bus; unique IDs address J1-J6. The computation platform and servo power paths are sized separately and share a reliable communication reference ground. A physical emergency-stop or readily accessible power disconnect is required.

J1 (ID 1) rotates the base, J2 (ID 2) drives the shoulder, J3 (ID 3) the elbow, J4 (ID 4) wrist pitch, J5 (ID 5) wrist roll, and J6 (ID 10) the gripper. J2 is a high-load joint and is explicitly forbidden from Motor mode. J1-J4 use bounded position control, whereas J5 and J6 use signed speed commands with release-to-stop behavior.

3.2 Bus Protocol and Kinematic Model

The implemented frame format is 0x55 0x55, servo ID, length, command, parameters, and checksum, where Length equals the number of parameter bytes plus three and Checksum = $\sim(\text{ID} + \text{Length} + \text{Command} + \text{Parameters}) \& 0xFF$. The parser verifies header, total length, ID, command, checksum, parameter length, and timeout before returning data. Position commands encode target and run time as little-endian 16-bit values; Motor commands encode mode, turn mode, and signed speed.

The simplified forward model treats J1 as yaw about base +Z, J2-J4 as a planar pitch chain, J5 as roll that changes orientation but not TCP position, and J6 as gripper state only. Measured dimensions are $h = 0.105$ m, $L_{23} = 0.130$ m, $L_{34} = 0.130$ m, $L_{45} = 0.060$ m, and $L_{5T} = 0.102$ m. A central-difference translational Jacobian and damped least-squares update map Cartesian or visual corrections to bounded joint increments.

3.3 Implemented Software Baseline

Component	Implemented function
bus_servo.py	Frame construction/parsing; position and Motor commands; status reads
joint_controller.py	YAML limits, angle mapping, run-time planning, command execution
arm_model.py	Forward kinematics, TCP orientation, numerical Jacobian
operation_terminal.py	Windows Cartesian steps, J5/J6 controls, Home, stop-all, dry-run
Ubuntu terminal	Independent serial discovery and equivalent manual controls
ROS 2 description	URDF/Xacro, RViz display, J6 mimic mapping
Perception/model/auto-task stack	Detector, backend adapter, visual servo, state-machine integration

This implementation status is deliberately conservative: experimental RGB-D and red-block scripts are not counted as a completed RGB closed-loop product path.

4. Closed-Loop Task Execution and Safety

4.1 Task State Machine

The designed executor contains IDLE, OBSERVE, PLAN, ALIGN_GRASP, GRASP, VERIFY_GRASP, LIFT, ALIGN_PLACE, DESCEND_PLACE, RELEASE, VERIFY_PLACE, RETURN_HOME, and ABORT states. Each state defines entry conditions, permitted actions, success criteria, a timeout, and a failure destination. ALIGN_GRASP and ALIGN_PLACE are iterative; GRASP and RELEASE require J6 position feedback; the two verification states require stable multi-frame visual evidence. RETURN_HOME is entered only after normal completion or when a recovery path is explicitly known to be safe.

4.2 Grasp Loop

The controller detects the selected object and both gripper tips, then minimizes the object's image displacement from the gripper centerline using bounded J1-J4 steps. A grasp is enabled only after the position and orientation

errors remain below their thresholds for consecutive frames. J6 then closes at a low speed while its position is polled. If the reported position remains within a configured tolerance for two seconds, contact is inferred; a short higher-speed command increases holding force and is followed by a stop command. After lifting, vision must confirm that the target moves with the gripper. This prevents an empty close or a dropped object from being reported as success.

4.3 Placement and Release Loop

Placement remains closed loop rather than ending after one open-loop move. At a safe pre-placement height the system detects the destination, gripper, and held object, then corrects horizontal and orientation error. It descends in small increments and reacquires the scene after every step. Once the release position is satisfied, J6 opens with negative speed and stops at the configured open position or timeout. A final multi-frame check requires the object to have left the gripper, lie within the destination region, and remain stable. Recoverable offsets trigger renewed observation or alignment; adhesion, loss of feedback, or an unsafe geometry causes a controlled stop.

4.4 Safety Boundary

Safety is layered across model output, planning, communication, and physical operation. Invalid JSON, unknown actions, missing object IDs, out-of-range joints, commands below the table plane, stale frames, serial timeouts, checksum failures, and non-converging loops must halt execution. On startup the controller first reads all joint states without moving, then validates one joint at a time with small, low-speed steps. J5/J6 continuous-speed commands are always stopped on release, stop-all, timeout, or application exit. A physical emergency stop cuts servo power independently of software. After a fault, the system preserves logs and waits for inspection; it does not blindly return home when the path or feedback is uncertain.

5. Test Method and Results

Testing follows the order software logic, isolated hardware modules, visual performance, closed-loop control, and end-to-end tasks. The present evidence was reproduced with Python's unittest runner without a connected robot. The core suite completed 50 tests and the independent Ubuntu terminal completed 8 tests, for 58 passing tests in total.

Test group	Count	Result	Primary coverage
arm_model	10	Pass	Angle mapping, FK, Home pose, Jacobian, limits
bus_servo	6	Pass	Move/Motor frames, negative speed, status parsing
joint_controller	12	Pass	YAML, timing, modes, angle targets, executor
operation_terminal	5	Pass	Cartesian steps, joystick, J5/J6, stop-all
servo_limit_tester	6	Pass	2-s stability, direction, timeout stop
urdf_description	11	Pass	Axes, limits, geometry, FK, J6 mapping
Ubuntu terminal	8	Pass	Protocol, discovery, Cartesian and J5/J6 controls
Total	58	All pass	Software and simulated-control logic only

The tests demonstrate that packet encoding, configuration parsing, simplified kinematics, manual control, stop behavior, and URDF consistency meet the current software baseline. They do not demonstrate camera throughput, physical positioning accuracy, detection precision, convergence time, or task success. The following hardware campaign is required before final acceptance.

Hardware test	Method	Target	Current evidence
RGB throughput	60-s timestamp and drop analysis under three loads	600 x 480 at 60 fps	Pending

Hardware test	Method	Target	Current evidence
Detection	Independent labeled set; TP/FP/FN by condition	Precision > 95%	Pending
Gripper localization	Calibrated points; mean/RMSE/max error	Mean < 2 cm	Pending
TCP localization	Repeated XYZ measurements across workspace	Mean < 2 cm	Pending
Visual convergence	Grasp/place timing; mean, P95, maximum	< 3 s	Pending
End-to-end task	Pick, place, and stack stage timing	< 60 s	Pending
Fault injection	Invalid plan, limit, loss, serial fault, timeout	Safe stop/recovery	Software partial; hardware pending

Result interpretation will report sample count, repeated trials, measurement instruments, mean and worst-case errors, and failure examples. A conservative threshold that produces high precision but low recall will not be described as successful detection, and an acceptable mean error will not hide a large workspace-edge maximum.

6. Features, Limitations, and Conclusion

6.1 Distinguishing Features

Dual model backends. A common interface supports cloud reasoning, offline local inference, and pre-motion fallback without changing the executor or exposing raw hardware commands.

Locally enforced safety. Model output is treated as untrusted structured input. Schema, object, allowlist, joint, workspace, and communication checks run on the Intel Core Ultra platform.

Feedback-based grasp and placement. J6 position stability provides low-cost contact evidence, while multi-frame vision verifies lifting, alignment, release, and final placement without an additional force sensor.

Modular engineering baseline. Servo protocol, configuration, kinematics, operation terminals, ROS 2 description, model adaptation, perception, and task execution are separated by explicit interfaces, allowing the camera, model, or compute platform to be replaced independently.

6.2 Current Limitations and Next Steps

Camera intrinsics, eye-in-hand extrinsics, HSV thresholds, and the visual Jacobian have not yet been measured on the final hardware. The camera's actual support for 600 x 480 at 60 fps must be verified rather than inferred from nominal capability. The final local model, quantization method, and inference latency remain to be selected for the competition host. Monocular stacking height is sensitive to lighting, occlusion, and object size, while J2 load error, backlash, and structural compliance require physical characterization.

The next integration sequence is: calibrate the camera and collect RGB samples; implement orange gripper-tip detection and target tracking; identify visual Jacobians at representative poses; integrate single-target alignment; add J6 contact and lift verification; then integrate placement, release, and result verification. The pending hardware tables in the Chinese report must be replaced by raw measurements, statistics, images, and failure records before final claims are made.

6.3 Conclusion

Huiyu Lingbi defines a practical architecture for connecting natural-language and multimodal reasoning to a tabletop robotic arm without surrendering the local safety boundary. Structured tasks isolate probabilistic models from the servo bus; kinematics, bounded increments, servo feedback, and fresh RGB observations form the execution loop. The implemented communication, configuration, kinematics, manual-control, ROS 2, and test infrastructure provide a reproducible base for the remaining perception and autonomous task integration. The project currently proves a 58-test software baseline, not the final physical performance targets.